Scaling Up through Hard Work

Kunliang Ruan

November 3, 2026

Working draft

Abstract

Can sustained agent collaboration enable an inexpensive model to solve more software engineering tasks than a single agent at maximum reasoning effort? We study this question through two contributions: a revision of DeepSWE and the MAGI architecture. The revision addresses evaluation-method defects, incorrect evaluation items, and restrictions unsupported by requirements or applicable conventions. Controlled regrading with unchanged implementations confirms that these defects affect measured outcomes. MAGI organizes further work around an end-to-end Worker that investigates, implements, tests, and repairs. An independent Reviewer identifies gaps in planning and delivery; a persistent Planner assesses them and directs fresh Workers to continue on the same repository. This protocol combines independent scrutiny with context renewal to elicit more of the model's existing capability. Using DeepSeek-V4.1-Flash-1017, MAGI fully solves 93 of 113 tasks (82.30%) under the revised evaluation, compared with 82 (72.57%) for mini-swe-agent at maximum effort and 86 for OpenCode. Relative to the maximum-effort baseline, MAGI uses 4.30 times the output tokens at 3.51 times the estimated token cost. These initial results show higher completion at greater inference expenditure. Execution cases trace successful repairs, a repair-induced regression, and an additional review exchange that yields no repair.

1 Introduction

Capable, inexpensive models create an opportunity to spend more inference on each task in pursuit of higher completion. DeepSeek-V4.1-Flash-1017 highlights this opportunity through strong coding performance at low cost, including a DeepSWE v1.1 comparison using the mini-swe-v2 harness (DeepSeek, 2026). We ask whether organizing additional work can improve task completion with this model. Here, *scaling up* means increasing inference work per task with fixed model weights; the capability being measured belongs to the resulting engineering system.

The first step is to raise the model's reasoning-effort setting. In our single-agent setup, however, this direct adjustment ends at max. Further inference requires another way to organize work. We compare this direct adjustment with additional execution and review across contexts. More expenditure alone is insufficient: HarnessTax finds substantial cost differences between harnesses without commensurate completion gains in its tested settings (Pan et al., 2026). The research question is therefore how additional inference can become useful engineering work.

Our local OpenCode reruns provide a concrete motivation. Of 27 tasks missed initially, 14 pass on a subsequent attempt with the same model (Section 6.2). The model can therefore produce passing solutions for many tasks that its initial execution fails to complete. These reruns follow external failure judgments, so their cumulative coverage is not directly comparable to completion through autonomous multi-round correction. An autonomous system must recognize when further work is needed and avoid introducing new errors while revising. MAGI pursues this through sustained execution and scrutiny using general-purpose repository and shell tools, without adding domain-specific problem-solving skill packages or tool wrappers that encode specialized solution procedures.

We use *hard work* to mean investigation, implementation, verification, and reconsideration of the same delivery objective. MAGI combines independent scrutiny and development feedback with renewed execution contexts on a retained workspace. Research on test-time compute examines how inference allocation affects performance (Snell et al., 2024); here the engineering objective is to complete work that an initial attempt misses.

This investigation requires both useful continuation and reliable measurement. DeepSWE's multi-language repository tasks provide the engineering setting (Huang et al., 2026), but public audits identify substantial grading defects (Epoch AI, 2026). Valid solutions can be rejected, and additional self-tests can break evaluator integration. We therefore first repair and validate the measurement, then examine how MAGI organizes additional work.

Contributions and study sequence. We contribute DeepSWE revisions covering evaluation methodology, individual evaluation items, and restrictive grading of underspecified requirements; controlled regrading validates representative repairs (Section 2). We then present MAGI's design rationale and protocol for continuing work around a complete execution agent (Section 3). System comparisons test whether this organization reaches higher completion at greater expenditure (Sections 4 and 5); execution cases examine successful corrections, regressions, and unnecessary review exchanges (Section 6). These findings motivate tests of repeatability, continuation decisions, and transfer to longer tasks.

2 Revising DeepSWE

DeepSWE comprises 113 original tasks across 91 repositories and five languages (Huang et al., 2026). Concise requests require repository exploration, requirements interpretation, substantial multi-file changes, and preservation of existing behavior. These demands make it suitable for evaluating additional engineering work.

DeepSeek reports extensive coding-agent reinforcement learning across scaffolds, using DeepSWE to track progress (DeepSeek-AI, 2026). HarnessTax observes substantial cost differences with often similar completion in other coding evaluations (Pan et al., 2026). These findings motivate our hypothesis that sustained execution can elicit further capability from a well-trained model; they do not establish its advantage over procedural guidance. Benchmark clarifications remain shared evaluation materials, separate from MAGI's work-organization protocol.

2.1 Why the benchmark needs revision

DeepSeek reports 77.0% pass@1 at maximum reasoning effort for DeepSeek-V4.1-Flash-1017 using mini-swe-v2 (DeepSeek, 2026). Epoch AI's audit identifies false-negative grading problems in at least 23 of 113 tasks (20.35%), including 18 with evaluator failures after agents modify or add tests. It stopped after establishing its verdict, making this a lower bound on incidence (Epoch AI, 2026). The scale of these defects is comparable to the 23% remaining failure share in the official result. This raises the possibility that apparent saturation partly reflects limits of the evaluation. The figures are not a task-by-task match and do not establish a numerical ceiling.

Our controlled experiments establish the practical impact of specific defects. With production implementations unchanged, correcting test integration, result identities, or invalid expectations restores successful checks (Table 1). An agent can therefore perform useful work that the original evaluator fails to recognize. This is particularly relevant when additional effort produces more self-tests or a different valid solution. Repairing these measurement errors is necessary to measure improvements in task completion on the affected tasks.

We organize the revision into three categories: shared evaluation-method defects, incorrect individual evaluation items, and underspecified requirements with overly restrictive grading (Ruan,

2026a). For each, the governing criterion is agreement between the visible task, applicable standards or repository conventions, and the behavior being checked. Similar alignment problems motivate OpenAI's SWE-bench Pro audit and task refinement in SWE-Bench Pro Verified (OpenAI, 2026; Zheng et al., 2026). UTBoost addresses insufficient tests and result-parser defects (Yu et al., 2025); PatchDiff and TestJack audit apparently passing patches (Wang et al., 2026; Yao et al., 2026). These studies motivate checking both false rejection and false acceptance. Our DeepSWE-specific evidence comes from controlled regrading and the retained obligations described below.

2.2 Defects in evaluation methodology

These defects arise from the way tests are integrated or results are identified, so a correction can apply across multiple tasks.

Go test integration. Agent-authored and hidden Go tests can introduce the same package-level names. They may run independently but fail to compile when combined. Verifier replacement of a test file can also remove helpers used by another self-test. These interactions can penalize additional verification work without revealing a defect in the production implementation.

We add a consistent self-test isolation convention to all 35 Go tasks: new self-tests use reserved file, test-function, and helper-name prefixes, with shared helpers kept in the reserved files. We checked that these prefixes are absent from the hidden test patches. Existing production interfaces, functional requirements, and test assertions retain their meaning. In a Prometheus transactional-reload diagnostic, renaming only colliding self-test symbols, with production code unchanged, restores successful compilation and passes all 15 new-behavior and 82 preserved-behavior checks. This isolates the integration problem; the diagnostic is recorded separately from the original benchmark result.

JUnit test identities. Here JUnit refers to the report format used for matching test results. In pytest-based tasks, automatically generated parameter IDs can depend on table order. Adding configuration entries or extending exported constants can shift these IDs, making an original test appear missing or attaching its identifier to a different input.

Reviewing the 35 tasks using this reporting path identified 24 requiring identity stabilization. We pin reference parameter identities and restore their scored names during test collection, preserving duplicate occurrences. Added inputs receive separate names, and missing original cases remain missing. The repair preserves test inputs, assertions, and raw outcomes. Validation includes controls for reordered, added, and removed cases across the 24 tasks, and assertion replay of 95 retained submissions. In Vulture, the same submitted patch moves from 291/295 to 295/295 preserved-behavior checks after the identity repair (Table 1).

2.3 Defects in individual evaluation items

An individual assertion, expected result, or fixture can conflict with an explicit requirement, an applicable standard, or the repository's established contract. We correct the specific conflict while retaining the remaining behavioral checks.

For CSS shorthand expansion, the evaluator distinguishes the equivalent background-size spellings `auto` and `auto auto`. The CSS specification supplies `auto` for an omitted second component (W3C, 2024). We normalize this equivalence only in the affected assertions. The same retained mini-swe-agent patch then passes 79/79 new-behavior checks, compared with 74/79 originally. Other output fields remain subject to their existing comparisons.

For Helm array merging, the task explicitly requires a warning when an annotated path is missing from the default values. Two success fixtures specify array merge paths but omit the

corresponding values, while rejecting the required warning. Supplying empty arrays in those fixtures resolves the contradiction. With the submitted implementation unchanged, the affected preserved-behavior score rises from 10/12 to 12/12; all 47 new-behavior checks continue to pass. Negative fixtures for missing paths and non-array values remain in place.

Table 1: Controlled regrading of retained mini-swe-agent patches. Production implementations are unchanged. Each row reports the affected check group; these are evaluator checks, not additional independent model attempts.

Task	Check group	Original	Revised
Vulture cache	Preserved behavior	291/295	295/295
CSS shorthand	New behavior	74/79	79/79
Helm array merge	Preserved behavior	10/12	12/12

2.4 Underspecified requirements with overly restrictive evaluation

In this category, requirements are ambiguous or broad, and neither an existing contract nor an applicable convention determines a unique behavior. Evaluation nevertheless imposes a narrower choice. The problem is the gap between the information available to the agent and the constraints used to judge it.

Testem illustrates an unnecessary restriction. The request specifies a warning with a particular prefix and fallback behavior for invalid configuration. The hidden test additionally requires the English word `invalid` in the warning body, although neither the request nor existing diagnostic conventions prescribe that wording. We replace this lexical restriction with a non-empty diagnostic check, preserving the required warning call, prefix, and fallback. Controls confirm that alternative wording is accepted while absent warnings, wrong prefixes, and empty or whitespace-only bodies still fail.

Termenv illustrates a requirement needing clarification. When no visible text fits a truncation width, retaining or dropping leading zero-width content can produce the same visible width. The original request does not settle this choice, and the baseline repository has no prior truncation policy that does. We make the intended preference explicit: retain leading zero-width content permitted by the output profile. Existing control-sequence integrity and closure requirements remain intact. Together, these cases show two ways to align the specification and evaluation: remove an unsupported restriction or disclose an intended constraint in the request.

2.5 Revision scope and validation

The revision index records 24 commits with their rationale and patches. These counts describe changes, not disjoint sets of defective tasks: one method-level repair can cover many tasks, and a task can involve multiple categories. The public patches record the conflicting requirement, convention, or evaluation behavior and the reason for the change. Beyond the detailed cases above, repairs include rebuilding Eicrud's generated JavaScript before testing edited TypeScript, isolating nested pytest imports in Returns, aligning SQLFmt fixtures with established spacing, and accepting valid Markdown link targets in Obsidian. Further instruction changes clarify error contracts and notification behavior; the Numba environment note allows versions compatible with the supplied checkout. The index also includes implementation guidance, which is distinct from correcting a scoring defect.

Validation follows the type of change. Integration and identity controls check that valid tests remain discoverable and missing or failing tests remain detectable. Item-level controls check valid alternatives and retain rejection of behavior that violates the task. In the Obsidian link-target repair, the previously accepted bare-space destination is rejected because it does not form the required Markdown link. Thus the revision can lower as well as raise scores. Historical

results remain preserved; regrading an unchanged submission is recorded separately from a fresh execution under clarified instructions.

Section 4 describes how revisions enter the evaluation records.

3 MAGI: Organizing Additional Inference Work

3.1 Design rationale: useful continuation beyond an initial attempt

A further attempt can succeed where an earlier one failed, but autonomous continuation requires more than another opportunity to generate code. The system must recognize unfinished work, decide whether a proposed correction belongs to the task, and carry it out without losing existing progress. Review itself can be mistaken, and a repair can introduce new defects. Reasoning experiments likewise distinguish locating an error from repairing one whose location is supplied (Tyen et al., 2024); their results do not establish repository-level correction rates. The objective here is better final delivery, with continuation and stopping treated as engineering judgments.

MAGI assigns overlapping responsibility for the same delivery: a complete Worker investigates, implements, and tests; a Reviewer scrutinizes planning and delivery; a persistent Planner assesses findings and directs further work. New orders start fresh Workers on the retained repository, while planning, review, and formal history preserve outstanding obligations. The design combines independent scrutiny and development feedback, as explored in Adversarial Review (Qiu and Gill, 2026) and Anthropic's three-role practice (Rajasekaran, 2026), with retained-workspace context renewal studied in Fork-and-Flush (Cai et al., 2026).

The system-level hypothesis is that this organization can reach a higher completion level than the direct effort adjustment alone, at additional inference cost. The experiments test that outcome; the cases examine whether review findings lead to useful repairs, introduce defects, or generate work without a repair. They do not isolate the effects of individual components. Table 2 locates the design along two dimensions: the allocation of engineering responsibilities and the symmetry of reasoning roles. These dimensions can coexist; Section 7 discusses related methods.

Table 2: MAGI compared with two reference patterns along different design dimensions. The patterns can coexist in a system.

Dimension	Reference pattern	MAGI
Engineering responsibility	Functional orchestration assigns engineering functions to specialists.	An end-to-end Worker receives additional planning and independent scrutiny.
Reasoning relationship	Symmetric redundancy uses peer solvers to produce and revise candidate answers.	Asymmetric roles revisit the same work and feed judgments into continued execution.

3.2 Complete execution and independent review

The Planner maintains the delivery objective, issues orders specifying outcomes, scope, and necessary constraints, and tracks unfinished obligations. It handles user clarification and changes to the delivery target across rounds, and decides when work is complete. Repository investigation and implementation choices belong to the Worker. Within its assigned scope, the Worker can read and edit files, execute commands, test, and correct its work before submitting a factual delivery report.

Independent review here means a separate conversational context and assessment role using the same underlying model; it does not imply statistically independent errors. The Reviewer examines two relationships: requirements to planning, and orders to delivery. This scope matters

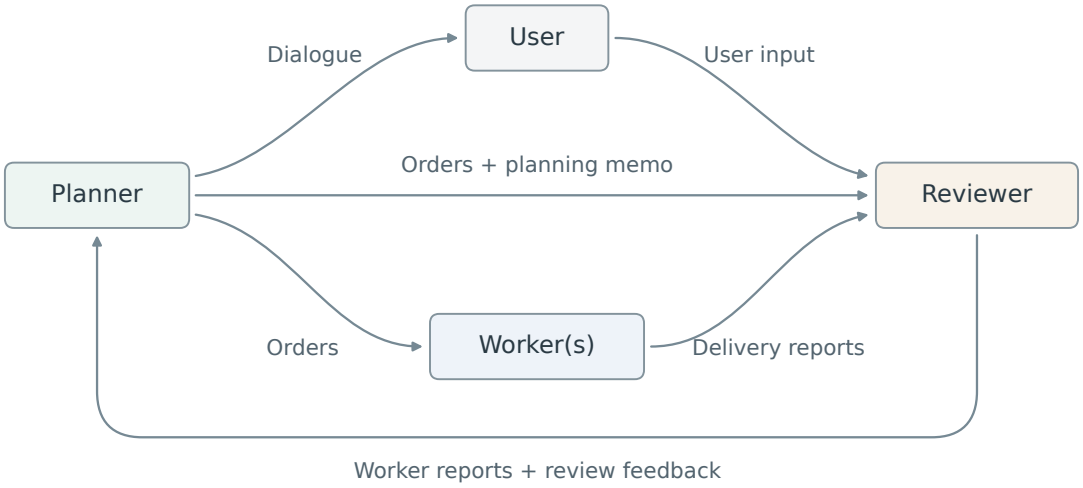


Figure 1: MAGI's communication structure sustains overlapping effort on the same delivery objective. Workers receive orders; the Reviewer also receives the planning memo, user input, and delivery reports. Original Worker reports and review feedback return to the Planner to guide subsequent work. Arrows show information flow; execution, planning, and review retain distinct responsibilities.

because a Worker can faithfully execute an order that omits part of the user's request. Reviewing planning keeps the order itself open to scrutiny, while reviewing delivery examines whether the reported work actually fulfills it. Tests contribute evidence to this judgment without defining the entire delivery obligation. The Reviewer can make targeted read-only repository queries to assess concrete concerns in reported delivery; it does not implement changes, run tests, or prescribe repair steps.

Information boundaries support these responsibilities (Figure 1). The Planner sends execution orders to Workers and a separate planning memo to the Reviewer. Workers receive the necessary outcomes and constraints, while planning assumptions remain available for review without automatically becoming implementation instructions. The Reviewer receives orders, the memo, user input, and Worker reports. Its findings return to the Planner alongside the original delivery reports, preserving the execution evidence and the assessment as distinct inputs to further planning.

3.3 Turning feedback into further work

Each round begins with orders and a planning memo. Workers execute the admitted orders concurrently with a separate Planner branch for user interaction. After these branches finish, the Reviewer examines their reports and the planning material. Both delivery reports and review findings enter the next planning round.

The Planner first considers whether a concern belongs to the delivery objective and then whether its factual basis establishes a gap. Disagreements with the Reviewer are discussed through the memo before new work is dispatched. Where the scope is agreed but decisive facts are missing, the Planner can assign a bounded investigation. Established gaps lead to correction orders, with the Worker retaining responsibility for implementation choices. A review therefore has a path into continued work without becoming an executable pass/fail gate.

Tool-informed feedback provides precedent for grounding revision in evidence (Gou et al., 2024). MAGI keeps implementation and testing with the Worker, while the Planner turns accepted findings into further orders. This loop applies even to one complete assignment. Larger tasks may use compatible concurrent orders, respecting write conflicts and dependencies. Decomposition is optional; the Planner closes the task when delivery obligations are satisfied.

3.4 Preserving progress across fresh Workers

Planner and Reviewer contexts persist across rounds, maintaining requirements, progress, and unresolved judgments. Each order starts a fresh Worker context, which is discarded after its delivery report. Repository changes persist, and formal records of user inputs, orders, reports, and reviews remain available through historical references. Applying Fork-and-Flush's renewal principle at order boundaries (Cai et al., 2026) allows a new execution pass to reconsider accumulated work without inheriting the entire earlier conversation. Section 7 distinguishes this from scored branch selection.

3.5 Extensions for changing requirements and long tasks

After orders are frozen, a cloned Planner context handles user dialogue concurrently with execution; new requirements enter subsequent planning. Only actual dialogue and source references return to the persistent Planner, while the Reviewer receives a summary. Private reasoning and tool exchanges remain in the branch, and its compaction does not replace the main context. This limits incidental dialogue propagation while preserving requirement changes.

Planner and Reviewer independently compact their growing contexts, retaining fixed instructions and recent messages while summarizing older history. Their different inputs and compression points may let one role recover a detail lost by the other through feedback and historical references. Since compaction can omit details whose importance emerges later (Anthropic, 2025), this is a proposed recovery mechanism; benefits from dynamic requirements handling and complementary memory remain unmeasured.

4 Experimental Protocol

We compare five configurations on the same 113 tasks under the revised evaluation. Each system's completion, token use, and estimated costs cover these same tasks, including failures. We specify execution, scoring, and resource accounting before presenting the completion–expenditure results and their comparison context.

4.1 Configurations and completion criterion

The baselines are OpenCode 2.0.29 at high reasoning effort and mini-swe-agent 2.4.6 (v2 series) at high and max effort, using their native prompts and tools. OpenCode has web search disabled. These provider-facing effort settings do not imply equal computation across systems. Throughout the tables and figures, the unqualified names mini-swe-agent and OpenCode denote their high-effort configurations; the mini-swe-agent max control is labeled separately.

The standard MAGI configuration uses high effort for Planner and Worker calls, low effort for the Reviewer, and high effort for Full compaction. MAGI-low changes the Worker's effort setting to low. Engine revisions also differ between parts of these runs, so this is a configuration comparison rather than a perfectly isolated single-factor ablation.

All final experiments use DeepSeek-V4.1-Flash-1017. The archived configurations record the provider's flash API alias. MAGI receives requirements through its CLI interaction flow. Subsequent questions receive a fixed reply to follow the task, use established conventions or judgment for unspecified matters, and keep checking the evidence. It supplies no task-specific answers. The baselines receive the requirement directly. The standard scheduling setup uses eight shared concurrent task slots, with a two-CPU quota and 8 GiB of memory per task. Historical adjustments and recovery decisions are retained in the experiment records.

Scoring and result construction. A task is fully solved when all required new-behavior and preserved-behavior checks pass. The effective-result ledger incorporates grading revisions and

recorded recovery decisions; it is not a simultaneous rerun on one final benchmark commit. It accepts an interrupted Koota snapshot run with full functional scores and replaces an earlier full-score Oxvg run with a completed, full-score independent rerun. The replacement leaves the solved count unchanged. The original-task mini-swe-agent comparison combines 41 new executions, 29 regradings of existing patches, and 43 equivalent reused results; one grading timeout remains in the denominator. These are maintained results, rather than an entirely unselected pass@1 experiment. Failure-only reruns are reported separately.

4.2 Token and cost accounting

For each task, token usage belongs to the same effective execution that supplies its completion result. All 113 tasks contribute to the totals. Input tokens include cache hits; output tokens include reasoning. Continuations use the execution's final cumulative usage, avoiding overlap between snapshots. For the original-task mini-swe-agent ledger, we reconstruct solving costs from 41 fresh runs and the existing runs behind 29 regraded patches and 43 reused results. All 29 replayed patches match the costed submissions byte for byte. Each solving run is counted once within that cohort; shared runs across original and revised cohorts are not additional project expenditure.

For cache-hit input I_h, cache-miss input I_m, and output O, the frozen price schedule yields the estimate

$$\widehat{C} = \frac{0.02I_h + I_m + 4O}{10^6} \quad \text{CNY}. \tag{1}$$

This is an estimated token cost for effective executions. It does not include separate failure-only reruns, diagnostic or replaced attempts, machine costs, task preparation, or external grading; it is not total project expenditure. Token totals and monetary cost are reported separately because caching changes the relationship between repeated input processing and price.

5 Results

5.1 Task completion beyond the direct effort adjustment

Table 3 shows MAGI fully solving 93 of 113 tasks (82.30%). The primary direct budget adjustment, increasing mini-swe-agent from high to max effort, raises its solved count from 81 to 82. MAGI completes 11 more tasks than this maximum-effort control. It also exceeds OpenCode's 86 solved tasks by seven. Relative to OpenCode and mini-swe-agent, the observed gains are 6.19 and 10.62 percentage points, respectively.

The gain comes with substantially more inference. Relative to mini-swe-agent max, the increase from 82 to 93 solved tasks uses 4.30 times the output tokens at 3.51 times the estimated token cost. Relative to OpenCode, the increase from 86 to 93 uses 15.65 times the output tokens at 6.37 times the cost. The corresponding ratios against mini-swe-agent are 5.66 and 4.43. Figure 2 shows the completion and expenditure of all five configurations. These observed gains demonstrate how additional organized work can improve task completion with the same inexpensive model. Appendix A lists the two DeepSWE score components for each task in the three main systems, with independent reruns on separate rows.

5.2 From reported scores to local system comparisons

Table 3 preserves two stages of comparison: establishing the local revised baseline, then comparing systems under that evaluation. The official high result is read from the published chart; only the max point is explicitly labeled 77.0% (DeepSeek, 2026).

The roughly nine-percentage-point gap between official and local original-task results remains unexplained. The named effort setting and task set match, but full execution and scoring configurations have not been matched. The V4.1 report specifies eight samples per DeepSWE

Table 3: Comparison sequence and inference use for DeepSeek-V4.1-Flash-1017. Groups distinguish original-task references, the revised baseline, system comparisons, and MAGI-low. The official high result is a chart estimate (DeepSeek, 2026); unqualified baseline names denote high effort. Resource totals cover all 113 tasks, including failures. Original-task mini-swe-agent costs are reconstructed from the executions behind its effective results (Section 4); dashes mark unavailable official resource totals. M denotes one million tokens; input includes cache hits and output includes reasoning. Costs follow Equation 1.

System / source	Evaluation	Fully solved	Input (M)	Output (M)	Cost (CNY)
DeepSeek report	Original	≈73%	—	—	—
mini-swe-agent	Original	72/113 (63.72%)	958.02	6.78	56.78
mini-swe-agent	Revised	81/113 (71.68%)	929.28	6.74	55.96
mini-swe-agent max	Revised	82/113 (72.57%)	1170.77	8.88	70.55
OpenCode	Revised	86/113 (76.11%)	981.71	2.44	38.89
MAGI	Revised	**93/113 (82.30%)**	2935.40	38.14	247.69
MAGI-low	Revised	85/113 (75.22%)	2061.15	34.09	213.14

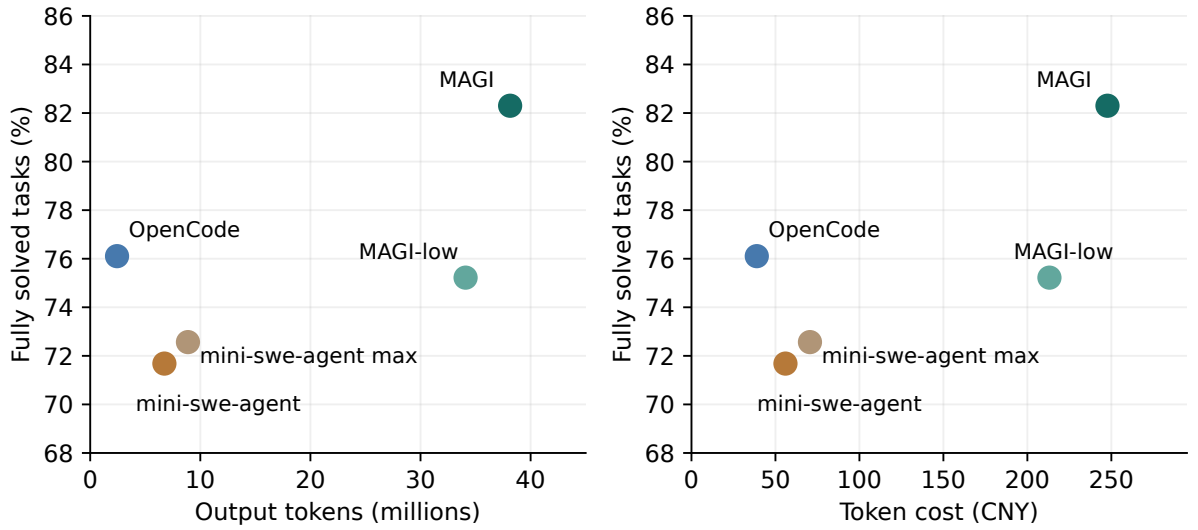


Figure 2: Completion versus output tokens and estimated cost on the same 113 tasks. MAGI occupies a higher-completion, higher-expenditure operating point. Both panels include unsuccessful tasks in their denominators and resource totals.

task in its cross-scaffold experiments (DeepSeek-AI, 2026); the 1017 release does not confirm that scheme or specify whether repeated full-run scores are averaged or selected. Official scores therefore remain reference points with an incompletely specified protocol.

The local increase from 72 to 81 combines regrading, clarified requirements, and fresh executions (Section 4); controlled checks in Table 1 isolate specific scoring effects. Closeness to the official score after revision does not resolve the original-task gap. MAGI's measured gains use the local revised baselines. Within that setting, high-to-max tests the direct effort control, the system comparisons examine work organization, and MAGI-low supplies a further configuration trade-off.

5.3 Where the additional effort goes

Of MAGI's estimated CNY 247.69 total, Workers account for CNY 158.52 (64.00%), the Planner for CNY 14.43 (5.83%), and the Reviewer for CNY 74.75 (30.18%). Compaction is charged to the role that owns the context, and interaction to the Planner. Most expenditure goes to

execution; planning and review together account for about 36%. The architecture therefore allocates inference both to producing engineering results and to judgments that can direct another execution pass.

Effort configurations. Raising mini-swe-agent from high to max increases output tokens by 31.75% for a net gain of one solved task. Reasoning accounts for 67.49% of generated output in the max cohort, showing substantial reasoning use. This is an output composition statistic, not a measure of context occupancy. MAGI spends further inference on execution and scrutiny across contexts, extending effort beyond what the direct reasoning setting controls.

MAGI-low solves 85 tasks at an estimated cost of CNY 213.14: eight fewer tasks at 14.0% lower cost than the standard configuration. This provides a second completion–cost point within MAGI. The engine differences described in Section 4 remain part of this configuration comparison.

6 Analysis of Continued Work

6.1 Autonomous correction in retained executions

The system comparison measures final outcomes; the following retained executions examine how continued work reaches them. Updo and Pebble show successful implementation and planning corrections, Vulture a repair-induced regression, and Valibot an extra review exchange without a repair. All four ultimately pass. These selected segments explain local processes, without estimating outcome frequencies or component effects. Their histories, patches, and tool-output excerpts are indexed in the accompanying evidence records.

Successful implementation repair: Updo. The policy-alerting task requires an SSL-expiry alert to fire once, then become eligible again after the certificate lifetime rises above a threshold and subsequently falls back. The first Worker reported passing self-tests, but the Reviewer found a missing interaction: an above-threshold sample was ignored when the check also emitted a target-state event, preventing the required reset. Self-tests had omitted concurrent state events.

The Planner accepted the finding and ordered a scoped repair. A fresh Worker added regression tests and changed the tracker to observe SSL samples on every check while preserving event precedence. Retained command outputs show all four concurrent-state subcases failing before the change and passing afterward. Final scores are 17/17 new-behavior and 123/123 preserved-behavior checks. Review thus supplied a concrete gap that became a tested correction.

Successful planning correction: Pebble. The durability-API task specifies that waiting for sequence number zero succeeds after any commit. The implementation instead required a durable commit, and the Planner held repairs pending clarification. After the fixed benchmark reply referred it back to the requirements, the Reviewer challenged the hold: the explicit zero exception could coexist with durability requirements for positive sequence numbers. It also withdrew an empty-batch concern after considering the Planner's repository-based explanation.

The Planner issued a correction order preserving the accepted empty-batch behavior. The next Worker implemented the zero exception and tested that a non-sync commit satisfies zero while positive numbers still require durability. Later review reconciled the documentation. Final scores are 59/59 new-behavior and 44/44 preserved-behavior checks. Here review corrected the decision about further work, as well as the interpretation governing implementation.

A regression introduced during repair: Vulture. The persistent-cache task requires reanalysis of modules that transitively import a changed file. After review identified missed relative imports, a Worker revised import extraction. That repair inadvertently stopped recording the

package target for `from . import *` and `from .. import *`: the new branch skipped the star alias without retaining the package dependency. The next review identified the regression, and the Planner ordered a further repair. Retained command outputs show both star-import cases failing because the importing module was reused after its package's `__init__.py` changed. After the package target was restored, the same five-test reproduction suite passed. The final evaluator records 24/24 new-behavior and 295/295 preserved-behavior checks. Thus a successful final delivery can conceal a repair-induced regression and the extra work needed to remove it. The old/new code establishes the lost dependency; no intermediate full-task evaluation establishes a complete-task pass-to-fail transition. This MAGI execution is distinct from the retained mini-swe-agent submission used to validate evaluator identities in Section 2.2.

An extra review exchange without a repair: Valibot. After a genuine recursion-depth limitation was repaired, the Reviewer raised a further concern about a placeholder embedded in a hand-written recursive TypeScript type alias inside a transform. The Planner dispatched no new Worker and challenged the scope: the request covered placeholders placed in composed schemas, not rewriting arbitrary recursive type annotations. On reconsideration, the Reviewer withdrew the finding and stated that no code, test, or investigation work was owed. This added planning–review exchange settled a disputed requirement but produced no implementation repair; the already-delivered depth fix remained unchanged. It illustrates the cost of an unsupported extension of the target and the value of rejecting it before further execution. It does not imply that verification without code changes is generally wasted, or quantify the share of unproductive rounds.

6.2 External failure selection and independent reruns

Table 4: Independent reruns selected using external failure judgments. Cumulative coverage means solved at least once in the maintained main result or the separate rerun. It is not an autonomous multi-round completion rate, a repeated-success rate, or a uniformly sampled pass@2 estimate.

System	Main solved	Rerun solved / attempted	Cumulative solved
OpenCode	86/113	14/27	100/113
MAGI	93/113	11/20	104/113

Table 4 considers additional attempts after external evaluation identifies failures. MAGI solves 11 of its 20 initially unsolved tasks on rerun, and OpenCode solves 14 of 27, extending observed coverage to 104 and 100 of 113 tasks, respectively. This substantial recovery indicates that the models can solve many tasks missed by their initial executions, providing evidence of capacity not realized in the initial attempt.

External selection versus autonomous correction. The external evaluator chooses which tasks receive another attempt. This feedback distinction is central to evaluating self-correction (Kamoi et al., 2024). Cumulative coverage retains all initially successful tasks and adds rerun successes, so it cannot decrease by construction. Autonomous correction must make its own continuation, revision, and stopping decisions. The preceding cases show why those decisions can produce useful repairs, new defects, or work without a repair. Failure-only reruns do not measure that balance. Their cumulative coverage cannot therefore be directly compared with autonomous multi-round completion or used to rank the two approaches, even at equal inference budgets.

For MAGI, the recovered tasks motivate investigating additional review–execution rounds together with better continuation and stopping criteria. This is an optimization hypothesis, not evidence that more autonomous rounds will yield the same recovery or monotonically improve

completion. Such a test must evaluate the workflow's final deliveries across all tasks, accounting for both successful repairs and newly introduced failures. The reruns also do not estimate repeated-run stability.

7 Related Work

Inference-time capability expansion. Test-time compute and agent scaling studies examine how resource allocation and organization affect performance (Snell et al., 2024; Kim et al., 2026). Repeated sampling expands successful-candidate coverage (Brown et al., 2024); Self-Refine iterates through model-generated feedback (Madaan et al., 2023), while Reflexion retains verbal reflections for subsequent attempts (Shinn et al., 2023). In test-guided code self-repair, gains depend on feedback quality and the cost of repair (Olausson et al., 2024). MAGI studies continued repository work with fixed model weights and an inexpensive model.

Independent scrutiny and development feedback. Adversarial Review (AR) can review code or plans, holding the artifact fixed while a Reviewer examines it and a Critic audits the review before revision (Qiu and Gill, 2026). MAGI shares this scrutiny of work and findings; its protocol gives the Planner continuing responsibility for requirements and follow-up orders across fresh Workers (Section 3).

AR reports 71.6% to 75.2% on SWE-bench Verified at approximately 4.5 times baseline tokens (Qiu and Gill, 2026). Its per-task token medians, versus our aggregate output totals, and different models and benchmarks preclude a direct efficiency ranking.

Anthropic's engineering report describes a planner, generator, and evaluator for long-running application development (Rajasekaran, 2026); its Opus 4.5 configuration uses continuous sessions with compaction. MAGI focuses on higher completion with an inexpensive model, using read-only review and Workers restarted per order. Both add planning and assessment around a capable executor.

Zhang et al. (2026) directly study verification as test-time scaling, comparing instructions, reminders, completion gates, and independent review. Their critic starts fresh and can run checks in the live workspace. MAGI's persistent Reviewer does not execute tests; the Planner adjudicates findings and directs fresh Workers. Independent review is thus a shared mechanism; our contribution concerns the combined protocol and its measured trade-off.

Context renewal and continuity. Fork-and-Flush preserves workspace results while renewing contexts and selecting scored continuations (Cai et al., 2026). MAGI adopts renewal at order boundaries, with persistent planning and review rather than scored branch selection. Other continuity mechanisms include progress files and repository history (Young, 2025), learned context-management actions (Liu et al., 2025), and recursive planning with Worker handoffs in Cursor's reported system (Lin, 2026).

Other organizations of additional effort. CodeR and ChatDev distribute engineering functions across specialized roles (Chen et al., 2024; Qian et al., 2024a). Self-Consistency aggregates sampled reasoning paths (Wang et al., 2023), while multiagent debate has peer solvers revise answers (Du et al., 2023). Choi et al. (2025) distinguish interaction from vote aggregation. SWE-Search evaluates and selects candidates within search (Antoniades et al., 2024). Asymmetric feedback also appears in MARS's independent reviews (Wang et al., 2025) and actor–critic supervision of proposed actions (Jiang et al., 2026). These dimensions can coexist. MAGI retains complete execution within each Worker and directs asymmetric judgments back into the same evolving work.

User interaction. Tell Me More! clarifies user intentions before execution (Qian et al., 2024b); Talker–Reasoner separates dialogue from slower reasoning through shared memory (Christakopoulou et al., 2024). Gander separates live interaction from background work and provides a read-only auxiliary-query fork (Orantqing et al., 2026). MAGI's interaction branch clones the planning context, then returns actual dialogue to the Planner and a source-linked summary to the Reviewer (Section 3.5).

8 Discussion and Conclusion

The study yields three findings. First, controlled checks show that evaluator defects can conceal valid engineering work, motivating DeepSWE's revision. Second, MAGI reaches higher observed completion at greater inference expenditure than the tested baselines. Third, retained executions show why autonomous continuation requires judgment: review can identify real gaps, a repair can regress, and a finding can exceed the task's scope. The resulting opportunity is to organize more useful work, rather than expect more rounds alone to improve delivery.

Scope of the evidence. The system comparison establishes an observed completion–expenditure trade-off. It does not isolate component contributions or establish equal-budget allocation efficiency. Time and funding limited this independent, part-time study to one full run for most configurations, with selective reruns and grading revisions recorded separately. The findings concern this model and task set; live requirements changes and recovery after compaction remain unvalidated.

Next tests motivated by the findings. The observed aggregate gain calls first for repeated full evaluations with frozen configurations and grading rules, to measure its stability and variation in expenditure. Equal-budget comparisons and component ablations would then distinguish the effects of more computation from its allocation.

The mixed local effects of review call for testing continuation and stopping criteria. Such experiments should evaluate final deliveries across all tasks, counting both successful repairs and newly introduced failures. Externally selected rerun successes motivate this work, but cannot predict the net benefit of autonomous correction (Section 6.2).

Finally, the current task scope calls for transfer tests. The planned Terminal-Bench 4.0 (TB4) evaluation was deferred for time and cost reasons; its resource calibration and task repairs reinforce the need to freeze versions and task-specific limits (Marten, 2026). Larger-project studies can test requirements continuity and longer histories, informed by ChainSWE's distinction between accumulated repository state and retained conversation (Jin et al., 2026). These comparisons should match model and tool/information access and jointly report completion, token cost, and elapsed time.

MAGI provides a concrete way to spend an inexpensive model's inference budget on continued engineering work; the DeepSWE revisions make the resulting deliveries more faithfully assessable.

Code and evaluation materials. The MAGI source code and DeepSWE revisions (v1.2-alpha) are publicly available (Ruan, 2026b,a). A local evidence package retains run identities, task ledgers, usage audits, prompt snapshots, and case records; its public release remains to be prepared.

References

Anthropic. Effective context engineering for AI agents, 2025. URL https://www.anthropic.com/research/effective-context-engineering-for-ai-agents. Engineering article, 2025-09-29; accessed 2026-10-30.

Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. SWE-Search: Enhancing software agents with monte carlo tree search and iterative refinement, 2024. URL https://arxiv.org/html/2410.20285v2. Version 2, 2024-10-29; architecture comparison only.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL https://arxiv.org/abs/2407.21787v3. Version 3, 2024-12-30.

Ziyang Cai, Christos Ziakas, Vasilis Kontonis, Tim Pearce, Siddhartha Sen, Akshay Krishnamurthy, Shivam Garg, and Dimitris Papailiopoulos. Fork-and-Flush: Escaping idea basins in autoresearch agents, 2026. URL https://arxiv.org/html/2610.07447v1. Version 1, 2026-10-05.

Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. CodeR: Issue resolving with multi-agent and task graphs, 2024. URL https://arxiv.org/html/2406.01304v1. Version 1, 2024-06-03; architecture comparison only.

Hyeong Kyu Choi, Xiaojin Zhu, and Sharon Li. Debate or vote: Which yields better decisions in multi-agent large language models?, 2025. URL https://arxiv.org/html/2508.17536v2. Version 2, 2025-10-23; NeurIPS 2025.

Konstantina Christakopoulou, Shibl Mourad, and Maja Matarić. Agents thinking fast and slow: A talker-reasoner architecture, 2024. URL https://arxiv.org/html/2410.08328v1. Version 1, 2024-10-10.

DeepSeek. DeepSeek-V4.1-Flash-1017: Wish-driven programming made real, 2026. URL https://api-docs.deepseek.com/news/news261017/. Official release, 2026-10-17; accessed 2026-10-30.

DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/blob/main/DeepSeek_V41_Tech_Report.pdf. Technical report; sections 5.1 and 5.3; accessed 2026-10-30.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL https://arxiv.org/abs/2305.14325.

Epoch AI. DeepSWE v1.1: Benchmark review, 2026. URL https://epoch.ai/benchmarks/deepswe/review. Review dated 2026-09-07; accessed 2026-10-30.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing, 2024. URL https://arxiv.org/html/2305.11738v4. ICLR 2024; arXiv version 4, 2024-02-21.

Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026. URL https://arxiv.org/abs/2607.07946.

Shuli Jiang, Zhaoyang Zhang, Yi Zhang, Shuo Yang, Wei Xia, and Stefano Soatto. Asymmetric actor–critic for multi-turn LLM agents, 2026. URL https://arxiv.org/html/2604.00304v1. Version 1, submitted 2026-03-31.

Qirui Jin, Lingching Tung, Kenan Li, Qiyang Shi, Yushi She, Huanzhong Jia, Harrison Zhao, Kejing Xia, Zhenbang Du, Jiaxin Pei, Zhenyu Zhang, Zhen Qi, Yuyan Duan, Wenke Lee, and Zijian Jin. ChainSWE: Benchmarking coding agents on multi-bug software maintenance, 2026. URL https://arxiv.org/html/2607.02606v2. Version 2, 2026-09-01; author list follows this version.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440, 2024. doi: 10.1162/tacl_a_00713. URL https://aclanthology.org/2024.tacl-1.78/.

Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a science of scaling agent systems, 2026. URL https://arxiv.org/html/2512.08296v3. Version 3, 2026-04-08; version explicitly selected for this manuscript.

Wilson Lin. Towards self-driving codebases, 2026. URL https://cursor.com/blog/self-driving-codebases. Cursor engineering report, 2026-02-05; accessed 2026-10-30.

Shukai Liu, Jian Yang, Bo Jiang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon SWE-agents, 2025. URL https://arxiv.org/html/2512.22087v1. Version 1, 2025-12-26.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback, 2023. URL https://arxiv.org/abs/2303.17651.

Ryan Marten. Terminal-Bench 4.0, 2026. URL https://www.tbench.ai/news/terminal-bench-4-0. Official release report. Accessed November 1, 2026.

Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=y0GJXRungR.

OpenAI. Separating signal from noise in coding evaluations, 2026. URL https://openai.com/index/separating-signal-from-noise-coding-evaluations/. Technical report, 2026-07-08; accessed 2026-11-03.

Orantqing, Shengpeng Ji, Junlong Tong, Jialong Zuo, Dongjie Fu, Di Cao, Yangzhuo Li, Shangda Wu, Franz, Evan, Theron Veyra, Changhao Pan, Jingyu Lu, Dongchao Yang, Zhifei Xie, Yang Tan, Xiaoyu Shen, Xiaoda Yang, Wenfu Wang, Teddy Sun, Steve Yves, and Zhou Zhao. Omni interaction agent technical report, 2026. URL https://arxiv.org/html/2609.08977v2. Version 2, 2026-09-09; title and author names follow this version.

Melissa Z. Pan, Shuo Yang, Negar Arabzadeh, Wei-Lin Chiang, Ion Stoica, and Matei Zaharia. HarnessTax: How much does the harness matter for coding agents?, 2026. URL https://harnesstax.github.io/. Author project report; accessed 2026-10-30.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development, 2024a. URL https://arxiv.org/html/2307.07924v5. Version 5, 2024-06-05; ACL 2024.

Cheng Qian, Bingxiang He, Zhong Zhuang, Jia Deng, Yujia Qin, Xin Cong, Zhong Zhang, Jie Zhou, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Tell me more! towards implicit user intention understanding of language model driven agents, 2024b. URL https://arxiv.org/html/2402.09205v1. Version 1, 2024-02-14.

Eric S. Qiu and Joyce Gill. Adversarial review: Structured disagreement for grounded agentic code review, 2026. URL https://arxiv.org/html/2608.18167v1. Version 1, 2026-08-16.

Prithvi Rajasekaran. Harness design for long-running application development, 2026. URL https://www.anthropic.com/research/harness-design-long-running-apps. Anthropic engineering report, 2026-03-24; accessed 2026-11-01.

Kunliang Ruan. DeepSWE benchmark revisions, 2026a. URL https://github.com/PeterRK/deep-swe/tree/d7d70aab86d70d378d4fd3ee754c01c4ab00dc65. Public revision tagged v1.2-alpha; per-run evaluation revisions recorded separately; accessed 2026-11-01.

Kunliang Ruan. MAGI: Source code, 2026b. URL https://github.com/PeterRK/magi. Public source repository; experimental build identities recorded separately; accessed 2026-11-01.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366v4. Version 4, 2023-10-10.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL https://arxiv.org/abs/2408.03314.

Gladys Tyen, Hassan Mansoor, Victor Cărbune, Peter Chen, and Tony Mak. LLMs cannot find reasoning errors, but can correct them given the error location. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13894–13908, 2024. doi: 10.18653/v1/2024.findings-acl.826. URL https://aclanthology.org/2024.findings-acl.826/.

W3C. CSS backgrounds and borders module level 3, 2024. URL https://www.w3.org/TR/2024/CRD-css-backgrounds-3-20240311/. Candidate Recommendation Draft, 2024-03-11; section 2.9; accessed 2026-10-30.

Xiao Wang, Jia Wang, Yijie Wang, Pengtao Dang, Sha Cao, and Chi Zhang. MARS: toward more efficient multi-agent collaboration for LLM reasoning, 2025. URL https://arxiv.org/html/2509.20502v1. Version 1, 2025-09-24.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023. URL https://arxiv.org/abs/2203.11171v4. Version 4, 2023-03-07; ICLR 2023.

You Wang, Michael Pradel, and Zhongxin Liu. Are "solved issues" in SWE-bench really solved correctly? an empirical study. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. doi: 10.1145/3744916.3764576. URL https://software-lab.org/publications/icse2026_SWE-bench-correctness.pdf.

Shuangjie Yao, Hao Wang, Koushik Sen, Simin Chen, Baishakhi Ray, and Dawn Song. TestJack: Should you trust the results in coding benchmarks? agentic coding benchmarks auditing via evaluator evolution, 2026. URL https://arxiv.org/html/2610.10619v1. Version 1, 2026-10-07.

Justin Young. Effective harnesses for long-running agents, 2025. URL https://www.anthropic.com/research/effective-harnesses-for-long-running-agents. Anthropic engineering report, 2025-11-26; accessed 2026-11-01.

Boxi Yu, Yuxuan Zhu, Pinjia He, and Daniel Kang. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3762–3774, 2025. doi: 10.18653/v1/2025.acl-long.189. URL https://aclanthology.org/2025.acl-long.189/.

Zhengxin Zhang, Erkang Zhu, Yin Lin, Claire Cardie, Bolin Ding, and Jingren Zhou. Rethinking verification in agent harnesses, 2026. URL https://arxiv.org/html/2610.34577v1. Version 1, 2026-10-25.

Pujun Zheng, Zixin Shang, Shufan Jiang, Wenhui Tian, Dongsheng Zhu, Zerun Ma, Dingbo Yuan, and Qi Zhang. SWE-Bench Pro Verified: A reliable benchmark for software engineering agents, 2026. URL https://arxiv.org/html/2609.08149v1. Version 1, 2026-09-08.

A Per-task DeepSWE Scores

Both baselines use high reasoning effort; MAGI uses its standard configuration. F2P (fail-to-pass) and P2P (pass-to-pass) show passed/total checks for new and preserved behavior. Grading timeouts are marked as errors. *Rerun* rows contain externally selected failure-only reruns; dashes mean no rerun. All results use the revised-task effective ledgers (Section 4).

Task	mini-swe-agent		OpenCode		MAGI	
	F2P	P2P	F2P	P2P	F2P	P2P
abs-module-cache-flags	20/20	3/3	20/20	3/3	20/20	3/3
abs-stepped-slices	6/6	6/6	6/6	6/6	6/6	6/6
actionlint-action-pinning-lint	55/55	145/145	55/55	145/145	55/55	145/145
adaptix-name-mapping-aliases	42/44	2738/2738	44/44	2738/2738	44/44	2738/2738
aiomonitor-task-snapshots-diff	53/53	8/8	53/53	8/8	53/53	8/8
anko-default-function-arguments	2/2	119/119	2/2	119/119	2/2	119/119
anko-typed-variable-bindings	7/9	94/94	9/9	94/94	7/9	94/94
Rerun		—		—	9/9	94/94
arcane-drift-detection-baselines	82/82	2/2	82/82	2/2	82/82	2/2
arktype-json-schema-refs-dependencies	23/25	1679/1679	23/25	1679/1679	25/25	1679/1679
Rerun		—	25/25	1679/1679		—
awilix-async-container-initialization	24/24	162/162	22/24	162/162	21/24	162/162
Rerun		—	23/24	162/162	24/24	162/162
bandit-incremental-cache-control	88/88	275/275	88/88	275/275	88/88	275/275
bandit-interprocedural-taint-checks	66/66	293/293	64/66	293/293	66/66	293/293
Rerun		—	64/66	293/293		—
bandit-structured-nosec-directives	69/69	282/282	69/69	282/282	67/69	281/282
Rerun		—		—	69/69	282/282
boa-hierarchical-evaluation-cancellation	17/17	7/7	17/17	7/7	17/17	7/7
cattrs-partial-structuring-recovery	69/69	7/7	69/69	7/7	69/69	7/7
clack-async-autocomplete-options	80/82	643/643	82/82	643/643	80/82	643/643
Rerun		—		—	78/82	643/643
claude-code-by-agents-recursive-delegation	7/7	31/31	6/7	31/31	7/7	31/31
Rerun		—	7/7	31/31		—
cliffy-config-file-parsing	37/37	451/451	37/37	451/451	37/37	451/451
csstree-shorthand-expansion-compression	79/79	16715/16715	79/79	16715/16715	79/79	16715/16715
dasel-html-document-format	144/146	1012/1012	140/146	1012/1012	144/146	1012/1012
Rerun		—	146/146	1012/1012	144/146	1012/1012
dateutil-rfc5545-timezone-interop	67/67	2035/2035	67/67	2035/2035	64/67	2035/2035
Rerun		—		—	67/67	2035/2035
drizzle-orm-window-function-builders	130/130	566/566	130/130	566/566	130/130	566/566
dynamodb-toolbox-conditional-attribute-requirements	31/31	1267/1267	31/31	1267/1267	31/31	1267/1267
dynamodb-toolbox-lazy-recursive-schemas	37/37	1267/1267	37/37	1267/1267	37/37	1267/1267
effect-sse-httpapi-streaming	46/47	70/70	47/47	70/70	47/47	70/70
eicrud-keyset-pagination-cursor	14/14	168/168	14/14	168/168	14/14	168/168
etree-xml-diff-patch	52/52	15/15	52/52	15/15	52/52	15/15
expr-try-catch-errors	79/79	66265/66265	79/79	66265/66265	78/79	66265/66265
Rerun		—		—	79/79	66265/66265
fastapi-deprecation-response-headers	136/137	3134/3134	137/137	3134/3134	137/137	3134/3134
fastapi-implicit-head-options	43/43	3134/3134	38/43	3134/3134	43/43	3134/3134
Rerun		—	43/43	3134/3134		—
fd-deterministic-multi-key-sorting	43/43	109/109	43/43	109/109	43/43	109/109
geo-shapeindex-serialization	24/24	599/599	24/24	599/599	24/24	599/599
go-critic-doc-link-checker	3/3	16/16	3/3	16/16	3/3	16/16
go-genai-streamed-function-args	6/6	62/62	6/6	62/62	6/6	62/62
go-git-worktree-merge-conflicts	17/17	2/2	17/17	2/2	17/17	2/2
goreleaser-retry-publish-auditing	29/29	29/29	29/29	29/29	29/29	29/29
gql-incremental-graphql-delivery	17/17	810/811	17/17	810/811	17/17	810/811
Rerun		—	17/17	810/811	15/17	810/811
happy-dom-abort-pending-body-reads	14/14	165/165	14/14	165/165	14/14	165/165
happy-dom-deterministic-intersectionobserver	14/14	9/9	13/14	9/9	13/14	9/9
Rerun		—	14/14	9/9	14/14	9/9
helm-array-merge-strategies	47/47	12/12	47/47	12/12	47/47	12/12
helm-unified-manifest-stream	5/5	2/2	5/5	2/2	5/5	2/2
httpx-deterministic-cookie-store	115/115	1281/1281	115/115	1281/1281	115/115	1281/1281
httpx-multipart-response-parsing	121/122	1272/1272	122/122	1272/1272	122/122	1272/1272
httpx-streaming-json-iteration	107/108	1404/1404	107/108	1404/1404	107/108	1404/1404
Rerun		—	108/108	1404/1404	108/108	1404/1404
igel-persist-feature-schema	23/24	2/2	24/24	2/2	24/24	2/2
ink-grid-box-layout	23/25	49/49	25/25	49/49	25/25	49/49
ipython-session-bundle-replay	0/17	0/29	17/17	29/29	17/17	29/29
katex-multicolumn-array-spans	94/94	599/599	94/94	599/599	94/94	599/599
kcp-go-multiplexed-kcp-streams	2/30	12/12	30/30	12/12	30/30	12/12
kea-atomic-signal-selectors	11/12	139/139	12/12	139/139	12/12	139/139
kgateway-consistent-hash-policy	2/2	214/214	2/2	214/214	2/2	214/214
kombu-single-active-consumer-priority	85/85	1421/1421	85/85	1421/1421	85/85	1421/1421
kombu-virtual-queue-dead-lettering	76/76	1412/1412	76/76	1412/1412	76/76	1412/1412
koota-composite-trait-aspects	47/51	172/172	51/51	172/172	51/51	172/172
koota-deferred-mutation-buffer	71/71	128/128	71/71	128/128	71/71	128/128
koota-entity-snapshot-rollback	83/84	47/47	84/84	47/47	84/84	47/47
koota-pair-relation-tracking	38/38	172/172	38/38	172/172	38/38	172/172
koota-query-predicates	41/43	172/172	43/43	172/172	43/43	172/172

Per-task scores (continued).

Task	mini-swe-agent		OpenCode		MAGI	
	F2P	P2P	F2P	P2P	F2P	P2P
kysely-window-grouping-helpers	254/254	22/22	184/254	22/22	254/254	22/22
Rerun		—	254/254	22/22		—
langchain-request-coalescing	50/50	232/232	50/50	232/232	50/50	232/232
mashumaro-flattened-dataclass-fields	66/66	30014/30014	66/66	30014/30014	66/66	30014/30014
meriyah-explicit-resource-declarations	47/47	51469/51469	47/47	51469/51469	47/47	51469/51469
mnamer-daemon-watch-lifecycle	51/51	319/319	50/51	319/319	51/51	319/319
Rerun		—	51/51	319/319		—
mobly-grouped-test-barriers	79/79	808/808	79/79	808/808	79/79	808/808
narwhals-rolling-window-suite	103/103	10093/10093	103/103	10093/10093	103/103	10093/10093
numba-stencil-boundary-modes	29/29	827/827	25/29	827/827	29/29	827/827
Rerun		—	29/29	827/827		—
obsidian-linter-auto-table-of-contents	38/41	1131/1131	35/41	1131/1131	40/41	1131/1131
Rerun		—	40/41	1131/1131	36/41	1131/1131
obsidian-linter-link-format-conversion	59/60	1131/1131	59/60	1131/1131	60/60	1131/1131
Rerun		—	59/60	1131/1131		—
obsidian-linter-scoped-ignore-markers	33/33	1133/1133	33/33	1133/1133	33/33	1133/1133
ofetch-per-origin-circuit-breaker	47/47	13/13	47/47	13/13	47/47	13/13
onedump-dump-encryption-pipeline	82/82	6/6	81/82	6/6	82/82	6/6
Rerun		—	76/82	6/6		—
opa-rego-rule-profiling	25/25	6/6	22/25	6/6	22/25	6/6
Rerun		—	22/25	6/6	25/25	6/6
opa-template-string-reconstruction	5/5	4/4	5/5	4/4	5/5	4/4
optique-conditional-option-dependencies	36/36	2034/2034	35/36	2034/2034	35/36	2034/2034
Rerun		—	35/36	2034/2034	35/36	2034/2034
oxvg-structural-selector-preservation	6/6	62/62	1/6	62/62	6/6	62/62
Rerun		—	1/6	62/62		—
participle-grammar-conflict-analysis	89/91	153/153	91/91	153/153	91/91	153/153
pebble-durability-wait-apis	59/59	44/44	57/59	44/44	59/59	44/44
Rerun		—	59/59	44/44		—
pest-character-class-coalescing	104/104	249/250	104/104	250/250	104/104	249/250
Rerun		—		—	101/104	250/250
prometheus-transactional-reload-status	15/15	82/82	15/15	82/82	15/15	82/82
prometheus-typed-label-sorting	17/17	28/28	17/17	28/28	17/17	28/28
psd-tools-blend-range-api	36/45	979/979	45/45	979/979	45/45	979/979
pwntools-tube-multiplexing	*Error: timeout*		73/73	1/1	73/73	1/1
python-statemachine-state-data-scoping	70/72	1286/1286	72/72	1286/1286	72/72	1286/1286
query-persist-restored-query-state	8/8	42/42	8/8	42/42	7/8	42/42
Rerun		—		—	7/8	42/42
quill-shared-toolbar-focus	13/13	22/22	13/13	22/22	13/13	22/22
returns-validated-error-accumulation	159/159	61/61	159/159	61/61	159/159	61/61
scc-bounded-memory-spilling	28/31	286/286	31/31	286/286	30/31	286/286
Rerun		—		—	28/31	286/286
scriggo-method-declarations	48/48	1049/1049	48/48	1049/1049	48/48	1049/1049
skrub-duration-encoding	130/130	2784/2784	130/130	2783/2784	130/130	2784/2784
Rerun		—	130/130	2784/2784		—
sql-formatter-bigquery-pipe-formatting	25/26	5709/5709	26/26	5709/5709	26/26	5709/5709
sqlfmt-create-table-ddl-formatting	28/32	1273/1273	27/32	1271/1273	32/32	1273/1273
Rerun		—	30/32	1271/1273		—
sqlite-utils-safe-import-checkpoints	58/60	1038/1038	60/60	1038/1038	60/60	1038/1038
superjson-error-stack-serialization	80/80	116/116	80/80	116/116	80/80	116/116
task-task-graph-export	20/20	17/17	20/20	17/17	19/20	17/17
Rerun		—		—	20/20	17/17
tengo-callable-instance-isolation	23/23	122/122	23/23	122/122	23/23	122/122
tengo-destructuring-bindings	91/91	132/132	91/91	132/132	88/91	132/132
Rerun		—		—	91/91	132/132
termenv-preserve-ansi-resets	35/35	87/87	35/35	87/87	35/35	87/87
testem-bail-on-test-failure	90/90	489/489	90/90	489/489	90/90	489/489
testem-per-launcher-reports	64/65	469/469	65/65	469/469	65/65	469/469
textual-kitty-key-phases	17/23	57/57	19/23	57/57	23/23	57/57
Rerun		—	19/23	57/57		—
textual-richlog-follow-state	19/20	6/6	19/20	6/6	14/20	6/6
Rerun		—	19/20	6/6	20/20	6/6
tomlkit-toml-table-converters	60/60	964/964	59/60	964/964	60/60	964/964
Rerun		—	60/60	964/964		—
true-myth-iterable-collection-combinators	96/96	561/561	94/96	561/561	96/96	561/561
Rerun		—	96/96	561/561		—
ts-pattern-match-each	85/85	6/6	85/85	6/6	85/85	6/6
updo-policy-alerting	17/17	123/123	16/17	123/123	17/17	123/123
Rerun		—	17/17	123/123		—
valibot-recursive-schema-composition	10/10	209/209	10/10	209/209	10/10	209/209
vitest-duration-sharding	56/56	24/24	56/56	24/24	56/56	24/24
vulture-persistent-analysis-cache	24/24	295/295	23/24	295/295	24/24	295/295
Rerun		—	23/24	295/295		—
wasmi-trap-coredumps	22/22	58/58	22/22	58/58	22/22	58/58
wazero-multi-module-snapshots	78/78	2/2	78/78	2/2	76/78	2/2
Rerun		—		—	73/78	2/2
yaegi-go-embed-directives	38/38	58/58	38/38	58/58	38/38	58/58
yjs-map-conflict-detection	9/9	231/231	9/9	231/231	9/9	231/231
ytt-jsonpath-query-api	103/103	1/1	103/103	1/1	103/103	1/1